

Instruction Selection

Chapter 9

Where We Are

Intermediate Representation

Chapter 8



Step #1: **IR tree** => a list of **canonical trees**
Step #2: Rearrange the **canonical trees** (into **traces**) so that every *CJUMP* is immediately followed by its false label.

Canonical Intermediate Representation

Chapter 9



Step #3: **Instruction Selection** – generate the **pseudo-assembly code** from the canonical trees in Step #2

Abstract Assembly Code

Chapter 10 & 11



Step #4: Perform **register allocations** on pseudo-assembly code

Assembly Code

The Back End: IR to Machine Code

- **Instruction Selection**

- Mapping IR into abstract assembly code

- **Register Allocation**

- Deciding which values will reside in registers
- Replace abstract registers with real registers

- **Instruction Scheduling** (This course does not cover)

- Reordering operations to hide latencies and exploit intraprocessor parallelism.

Instruction Selection: Why and What

- The intermediate representation (Tree) language expresses only one operation in each tree node.
 - fetch, store, addition, subtraction, conditional jump, ...
- A real machine instruction can often perform several of primitive operations.



- The **instruction selection** phase: Finding the appropriate machine instructions to implement a given intermediate representation tree.

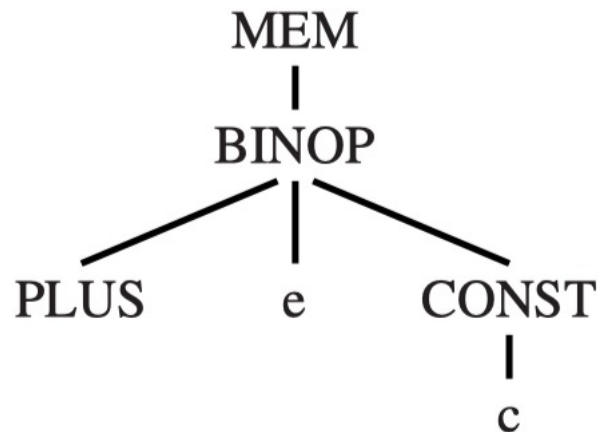
Instruction Selection: Implementation

Use **pattern matching techniques** to pick machine instructions that match fragments of the program IR.

- **Tree-oriented IR** suggests pattern matching on trees
 - Tree-patterns as input
 - E.g., Dynamic programming-based matching
- Linear IR suggests using some sort of string matching
 - Strings as input
 - E.g., Text matching, peephole matching, etc.

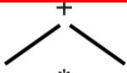
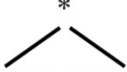
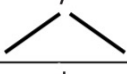
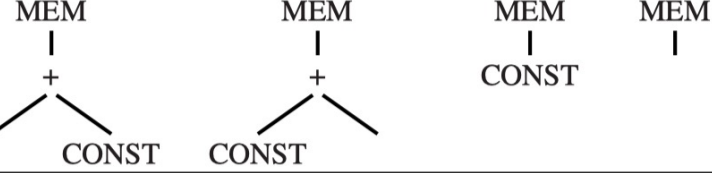
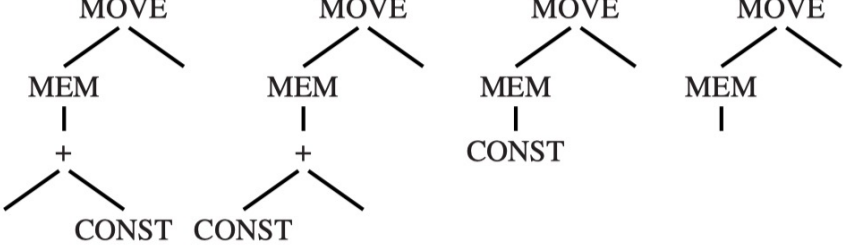
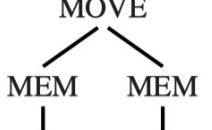
Tree Patterns

- Each machine instruction can be expressed a fragment of an IR tree, called a **tree pattern**.



- Instruction selection**: tiling the tree with a minimal set of tree patterns.
 - Tiling: cover the tree with nonoverlapping tree patterns
- To illustrate instruction selection, we use the **Jouette** architecture invented in the textbook

The Jouette Architecture

Name	Effect	Trees
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	
MUL	$r_i \leftarrow r_j \times r_k$	
SUB	$r_i \leftarrow r_j - r_k$	
DIV	$r_i \leftarrow r_j / r_k$	
ADDI	$r_i \leftarrow r_j + c$	
SUBI	$r_i \leftarrow r_j - c$	
LOAD	$r_i \leftarrow M[r_j + c]$	
STORE	$M[r_j + c] \leftarrow r_i$	
MOVEM	$M[r_j] \leftarrow M[r_i]$	

The Jouette Architecture

Name	Effect	Trees
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	$\begin{array}{c} + \\ \swarrow \quad \searrow \end{array}$
MUL	$r_i \leftarrow r_j \times r_k$	$\begin{array}{c} * \\ \swarrow \quad \searrow \end{array}$
SUB	$r_i \leftarrow r_j - r_k$	$\begin{array}{c} - \\ \swarrow \quad \searrow \end{array}$
DIV	$r_i \leftarrow r_j / r_k$	$\begin{array}{c} / \\ \swarrow \quad \searrow \end{array}$
ADDI	$r_i \leftarrow r_j + c$	$\begin{array}{c} + \\ \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
SUBI	$r_i \leftarrow r_j - c$	$\begin{array}{c} - \\ \swarrow \quad \searrow \\ \text{CONST} \end{array}$
LOAD	$r_i \leftarrow M[r_j + c]$	$\begin{array}{c} \text{MEM} \quad \text{MEM} \quad \text{MEM} \quad \text{MEM} \\ \quad \quad \quad \\ + \quad + \quad \text{CONST} \quad \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
STORE	$M[r_j + c] \leftarrow r_i$	$\begin{array}{c} \text{MOVE} \quad \text{MOVE} \quad \text{MOVE} \quad \text{MOVE} \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{MEM} \quad \text{MEM} \quad \text{MEM} \quad \text{MEM} \\ \quad \quad \quad \\ + \quad + \quad \text{CONST} \quad \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
MOVEM	$M[r_j] \leftarrow M[r_i]$	$\begin{array}{c} \text{MOVE} \\ \swarrow \quad \searrow \\ \text{MEM} \quad \text{MEM} \\ \quad \end{array}$

- BINOP(PLUS, x, y) $\Rightarrow$ +(x, y)

The Jouette Architecture

Name	Effect	Trees
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	$\begin{array}{c} + \\ \swarrow \quad \searrow \\ * \end{array}$
MUL	$r_i \leftarrow r_j \times r_k$	$\begin{array}{c} * \\ \swarrow \quad \searrow \\ - \end{array}$
SUB	$r_i \leftarrow r_j - r_k$	$\begin{array}{c} - \\ \swarrow \quad \searrow \\ / \end{array}$
DIV	$r_i \leftarrow r_j / r_k$	$\begin{array}{c} / \\ \swarrow \quad \searrow \\ + \end{array}$
ADDI	$r_i \leftarrow r_j + c$	$\begin{array}{c} + \\ \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
SUBI	$r_i \leftarrow r_j - c$	$\begin{array}{c} - \\ \swarrow \quad \searrow \\ \text{CONST} \end{array}$
LOAD	$r_i \leftarrow M[r_j + c]$	$\begin{array}{c} \text{MEM} \quad \text{MEM} \quad \text{MEM} \quad \text{MEM} \\ \quad \quad \quad \\ + \quad + \quad \text{CONST} \quad \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
STORE	$M[r_j + c] \leftarrow r_i$	$\begin{array}{c} \text{MOVE} \quad \text{MOVE} \quad \text{MOVE} \quad \text{MOVE} \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{MEM} \quad \text{MEM} \quad \text{MEM} \quad \text{MEM} \\ \quad \quad \quad \\ + \quad + \quad \text{CONST} \quad \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
MOVEM	$M[r_j] \leftarrow M[r_i]$	$\begin{array}{c} \text{MOVE} \\ \swarrow \quad \searrow \\ \text{MEM} \quad \text{MEM} \\ \quad \end{array}$

- BINOP(PLUS, x, y) => +(x, y)
- r0 = 0
- The actual values of CONST and TEMP nodes will not always be shown.

The Jouette Architecture

Name	Effect	Trees
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	$\begin{array}{c} + \\ \swarrow \quad \searrow \\ * \\ \swarrow \quad \searrow \end{array}$
MUL	$r_i \leftarrow r_j \times r_k$	$\begin{array}{c} * \\ \swarrow \quad \searrow \\ + \\ \swarrow \quad \searrow \end{array}$
SUB	$r_i \leftarrow r_j - r_k$	$\begin{array}{c} - \\ \swarrow \quad \searrow \\ / \\ \swarrow \quad \searrow \end{array}$
DIV	$r_i \leftarrow r_j / r_k$	$\begin{array}{c} / \\ \swarrow \quad \searrow \\ + \\ \swarrow \quad \searrow \end{array}$
ADDI	$r_i \leftarrow r_j + c$	$\begin{array}{c} + \\ \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
SUBI	$r_i \leftarrow r_j - c$	$\begin{array}{c} - \\ \swarrow \quad \searrow \\ \text{CONST} \end{array}$
LOAD	$r_i \leftarrow M[r_j + c]$	$\begin{array}{c} \text{MEM} \quad \text{MEM} \quad \text{MEM} \quad \text{MEM} \\ \quad \quad \quad \\ + \quad + \quad \text{CONST} \quad \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
STORE	$M[r_j + c] \leftarrow r_i$	$\begin{array}{c} \text{MOVE} \quad \text{MOVE} \quad \text{MOVE} \quad \text{MOVE} \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \quad \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{MEM} \quad \text{MEM} \quad \text{MEM} \quad \text{MEM} \\ \quad \quad \quad \\ + \quad + \quad \text{CONST} \quad \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
MOVEM	$M[r_j] \leftarrow M[r_i]$	$\begin{array}{c} \text{MOVE} \\ \swarrow \quad \searrow \\ \text{MEM} \quad \text{MEM} \\ \quad \end{array}$

- BINOP(PLUS, x, y) => +(x, y)
- r0 = 0
- The actual values of CONST and TEMP nodes will not always be shown.

The Jouette Architecture

Name	Effect	Trees
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	
MUL	$r_i \leftarrow r_j \times r_k$	
SUB	$r_i \leftarrow r_j - r_k$	
DIV	$r_i \leftarrow r_j / r_k$	
ADDI	$r_i \leftarrow r_j + c$	
SUBI	$r_i \leftarrow r_j - c$	
LOAD	$r_i \leftarrow M[r_j + c]$	
STORE	$M[r_j + c] \leftarrow r_i$	
MOVEM	$M[r_j] \leftarrow M[r_i]$	

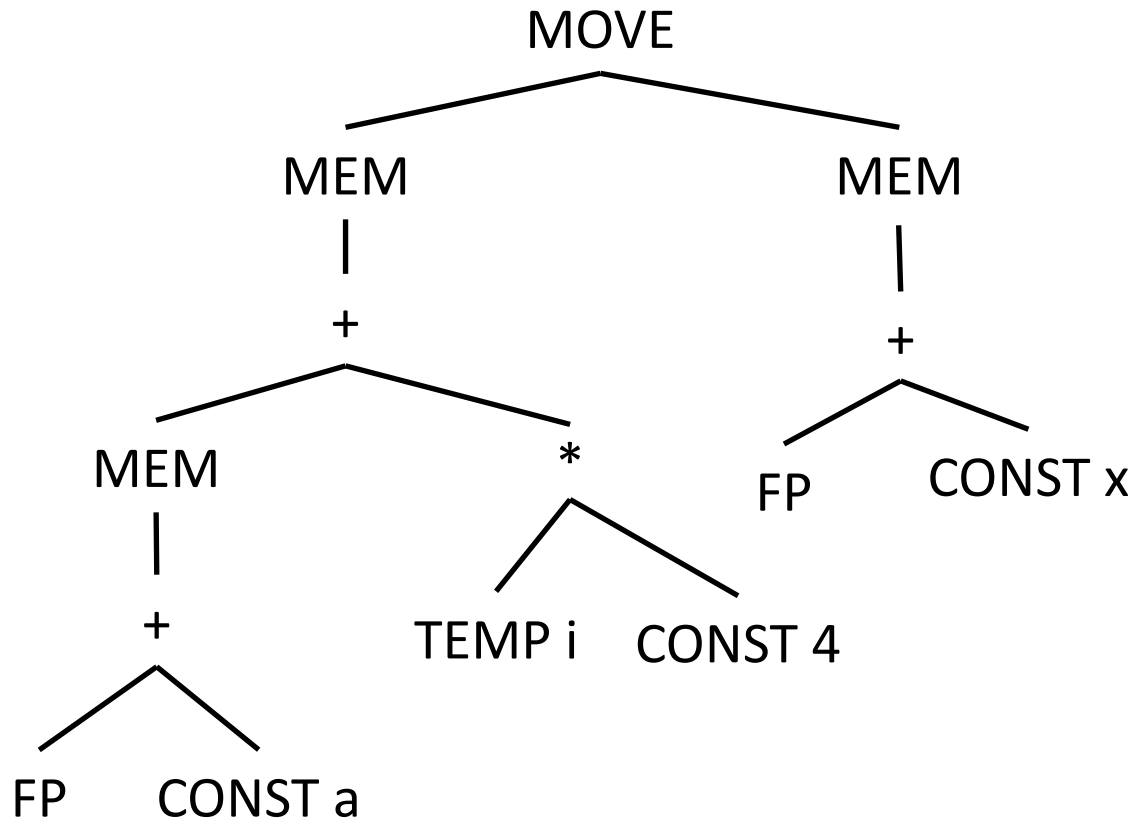
- BINOP(PLUS, x, y) => +(x, y)
- r0 = 0
- The actual values of CONST and TEMP nodes will not always be shown.

Specification about Jouette Architecture

- Register `r0` always contains zero
- Some instructions correspond to more than one tree pattern

Tree Patterns - Example

- The fundamental idea of instruction selection using a tree-based IR is *tiling* the IR tree.
- The tiles are the set of tree patterns corresponding to legal machine instructions
- For example: $a[i] := x$

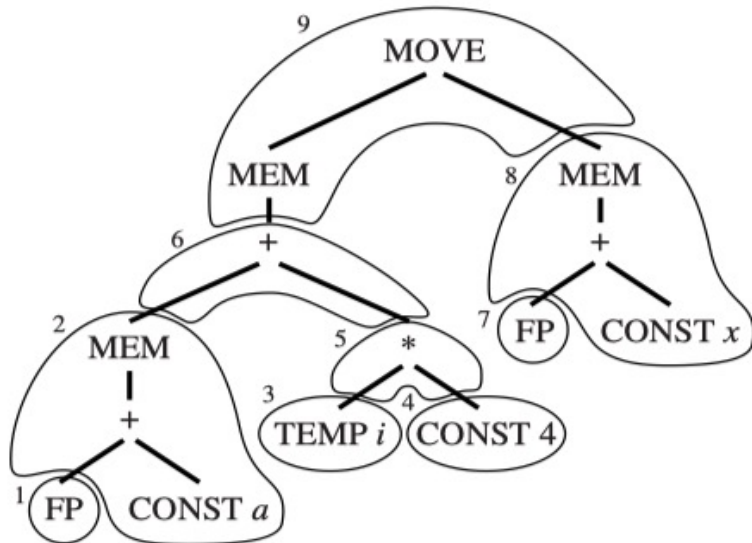


Last Week

- Instruction selection: why and what
- Tree Patterns
- The Jouette architecture

Tree Patterns - Example

- The tree of $a[i] := x$ can be tiled as follows:



2	LOAD	$r_1 \leftarrow M[\mathbf{fp} + a]$
4	ADDI	$r_2 \leftarrow r_0 + 4$
5	MUL	$r_2 \leftarrow r_i \times r_2$
6	ADD	$r_1 \leftarrow r_1 + r_2$
8	LOAD	$r_2 \leftarrow M[\mathbf{fp} + x]$
9	STORE	$M[r_1 + 0] \leftarrow r_2$

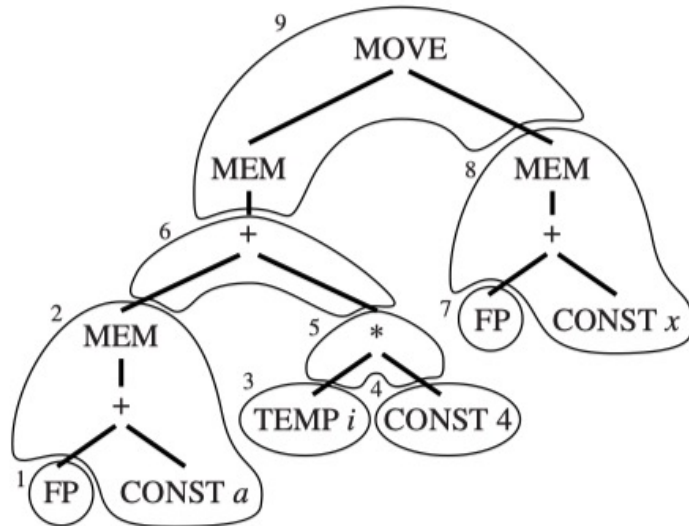
(a)

Name	Effect	Trees
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	$\begin{array}{c} + \\ \swarrow \quad \searrow \end{array}$
MUL	$r_i \leftarrow r_j \times r_k$	$\begin{array}{c} * \\ \swarrow \quad \searrow \end{array}$
SUB	$r_i \leftarrow r_j - r_k$	$\begin{array}{c} - \\ \swarrow \quad \searrow \end{array}$
DIV	$r_i \leftarrow r_j / r_k$	$\begin{array}{c} / \\ \swarrow \quad \searrow \end{array}$
ADDI	$r_i \leftarrow r_j + c$	$\begin{array}{c} + \\ \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
SUBI	$r_i \leftarrow r_j - c$	$\begin{array}{c} - \\ \swarrow \quad \searrow \\ \text{CONST} \end{array}$
LOAD	$r_i \leftarrow M[r_j + c]$	$\begin{array}{c} \text{MEM} \quad \text{MEM} \quad \text{MEM} \quad \text{MEM} \\ \quad \quad \quad \\ + \quad + \quad \text{CONST} \quad \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
STORE	$M[r_j + c] \leftarrow r_i$	$\begin{array}{c} \text{MOVE} \quad \text{MOVE} \quad \text{MOVE} \quad \text{MOVE} \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{MEM} \quad \text{MEM} \quad \text{MEM} \quad \text{MEM} \\ \quad \quad \quad \\ + \quad + \quad \text{CONST} \quad \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
MOVEM	$M[r_j] \leftarrow M[r_i]$	$\begin{array}{c} \text{MOVE} \\ \swarrow \quad \searrow \\ \text{MEM} \quad \text{MEM} \\ \quad \end{array}$

- Tiles 1, 3, and 7 do not correspond to any machine instructions, because they are just registers (TEMPs).

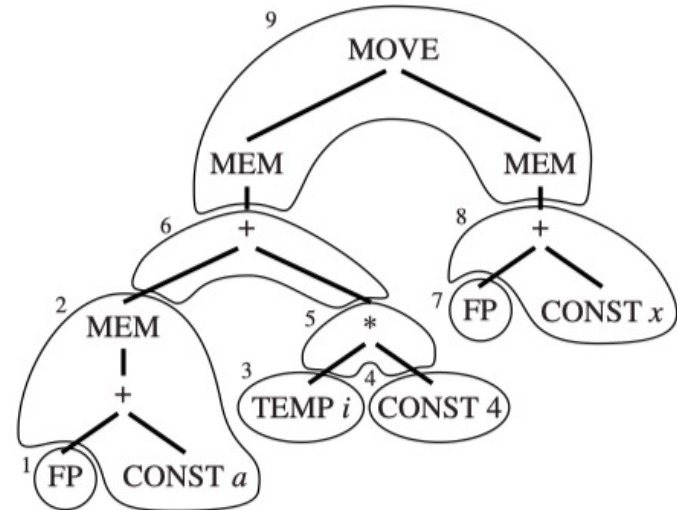
Tree Patterns - Example

- The tree of $a[i] := x$ tiled in two ways:



2	LOAD	$r_1 \leftarrow M[\mathbf{fp} + a]$
4	ADDI	$r_2 \leftarrow r_0 + 4$
5	MUL	$r_2 \leftarrow r_i \times r_2$
6	ADD	$r_1 \leftarrow r_1 + r_2$
8	LOAD	$r_2 \leftarrow M[\mathbf{fp} + x]$
9	STORE	$M[r_1 + 0] \leftarrow r_2$

(a)



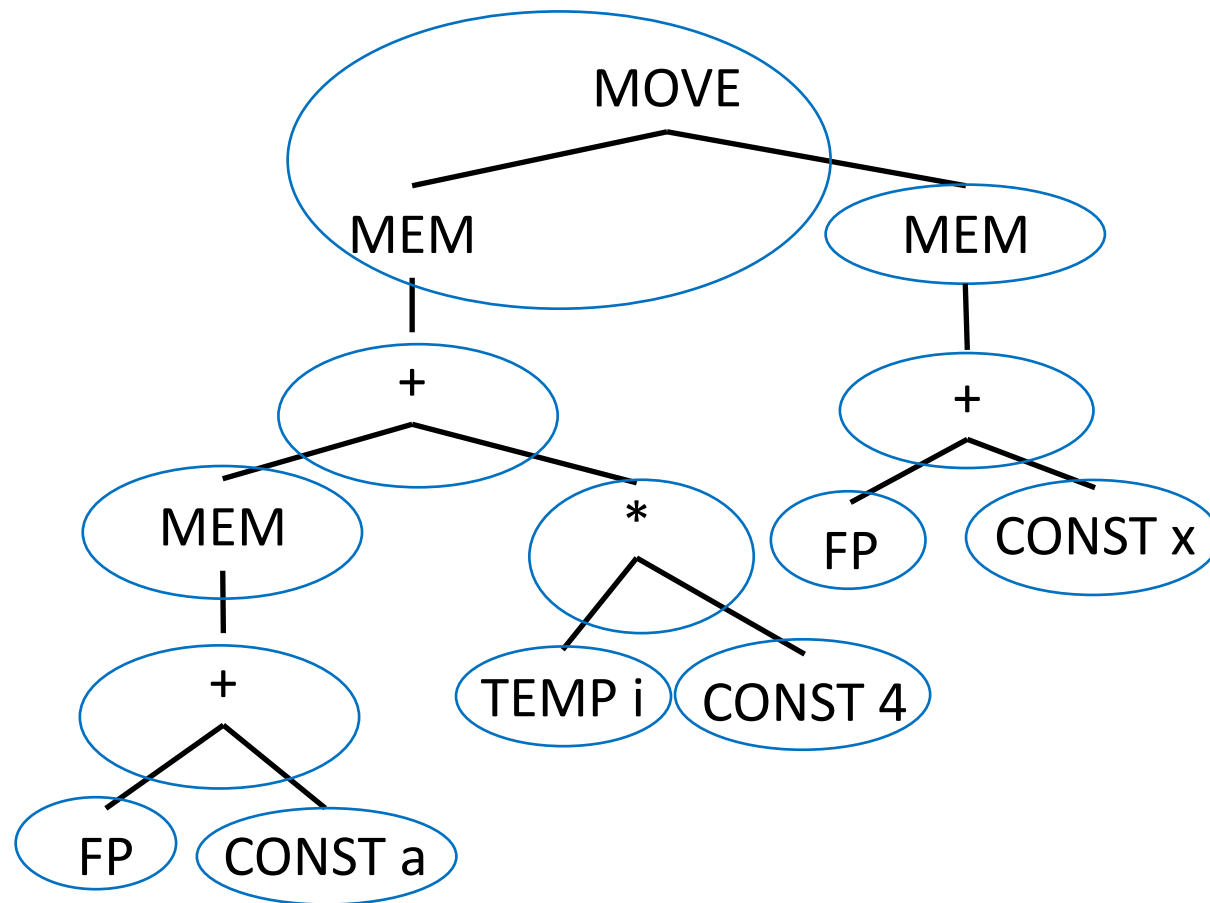
2	LOAD	$r_1 \leftarrow M[\mathbf{fp} + a]$
4	ADDI	$r_2 \leftarrow r_0 + 4$
5	MUL	$r_2 \leftarrow r_i \times r_2$
6	ADD	$r_1 \leftarrow r_1 + r_2$
8	ADDI	$r_2 \leftarrow \mathbf{fp} + x$
9	MOVEM	$M[r_1] \leftarrow M[r_2]$

(b)

- Tiles 1, 3, and 7 do not correspond to any machine instructions, because they are just registers (TEMPs).

Tree Patterns -Example

- It is always possible to tile the tree with tiny tiles, each covering only one node.
- For $a[i] := x$, such a tiling looks like this:



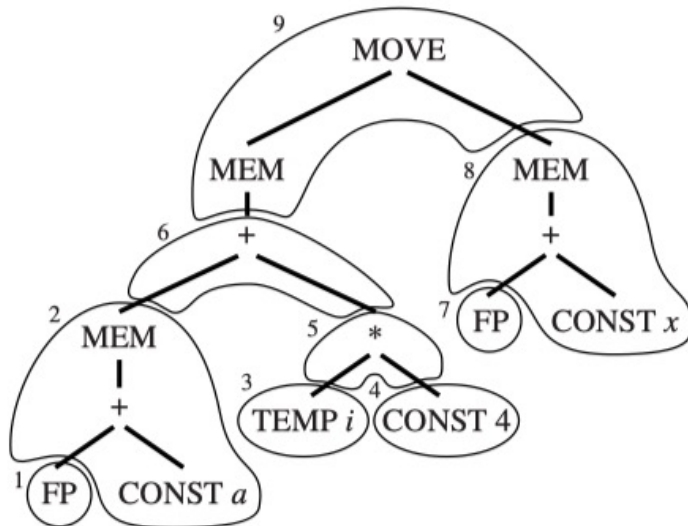
ADDI	$r1 \leftarrow r0 + a$
ADD	$r1 \leftarrow fp + r1$
LOAD	$r1 \leftarrow M[r1 + 0]$
ADDI	$r2 \leftarrow r0 + 4$
MUL	$r2 \leftarrow ri \times r2$
ADD	$r1 \leftarrow r1 + r2$
ADDI	$r2 \leftarrow r0 + x$
ADD	$r2 \leftarrow fp + r2$
LOAD	$r2 \leftarrow M[r2 + 0]$
STORE	$M[r1 + 0] \leftarrow r2$

Optimal and Optimum Tilings

- An IR tree can be tiled in multiple ways.
- **Best tiling**: an instruction sequence of least cost:
 - For single issue fixed-latency machine, meaning fewest number of instructions.
- **Optimum tiling**: the one whose tiles sum to the lowest possible value.
 - Globally “the best”
- **Optimal tiling**: the one where no two adjacent tiles can be combined into a single tile of lower cost.
 - Locally “the best”
 - If there is some tree pattern that can be split into several tiles of lower combined cost, then we should remove that pattern from our catalog of tiles.
- **Every optimum tiling is also optimal, but not vice versa.**

Optimal and Optimum Tilings - Example

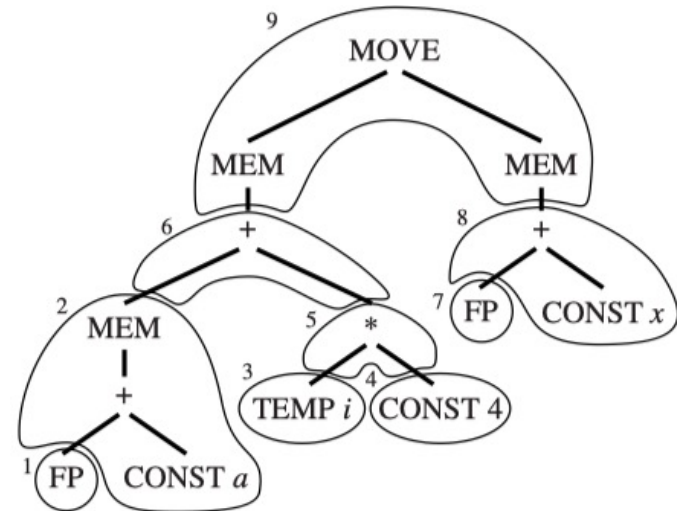
- Suppose every instruction costs **one** unit, except for **MOVEM** which costs **m** units.



2	LOAD	$r_1 \leftarrow M[\mathbf{fp} + a]$
4	ADDI	$r_2 \leftarrow r_0 + 4$
5	MUL	$r_2 \leftarrow r_i \times r_2$
6	ADD	$r_1 \leftarrow r_1 + r_2$
8	LOAD	$r_2 \leftarrow M[\mathbf{fp} + x]$
9	STORE	$M[r_1 + 0] \leftarrow r_2$

(a)

6 units



2	LOAD	$r_1 \leftarrow M[\mathbf{fp} + a]$
4	ADDI	$r_2 \leftarrow r_0 + 4$
5	MUL	$r_2 \leftarrow r_i \times r_2$
6	ADD	$r_1 \leftarrow r_1 + r_2$
8	ADDI	$r_2 \leftarrow \mathbf{fp} + x$
9	MOVEM	$M[r_1] \leftarrow M[r_2]$

(b)

5+m units

Optimum? Optimal?

Outline

- Algorithms for Instruction Selection
 - Maximal Munch
 - Dynamic Programming
 - Tree Grammar
- CISC Machines
- Instruction Selection for the Tiger Compiler

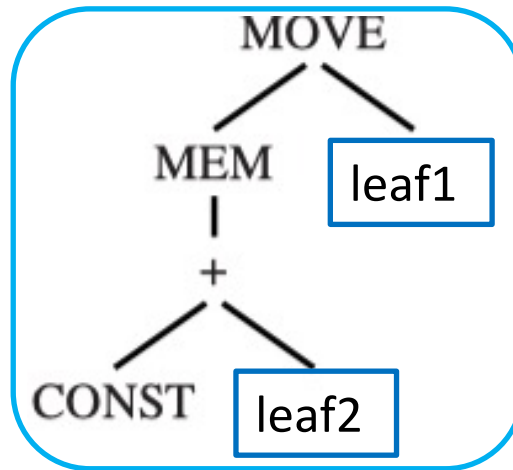
Outline

- Algorithms for Instruction Selection
 - **Maximal Munch**
 - Dynamic Programming
 - Tree Grammar
- CISC Machines
- Instruction Selection for the Tiger Compiler

Maximal Munch

Maximal Munch: the algorithm for optimal tiling

- Assume larger tiles = better
- **Main idea**: Greedy algorithm
 - Top-down Strategy
 - Cover the current node with the largest matchable tile
 - Repeat on subtrees attached to the leaves of the tile



Maximal Munch

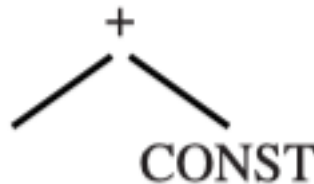
- **Overall Procedure:**

- Starting at the root of the tree, find the largest tile that fits.
- Cover the root node – and perhaps several other nodes near the root – with this tile, leaving several subtrees.
- Repeat the same algorithm for each subtree.

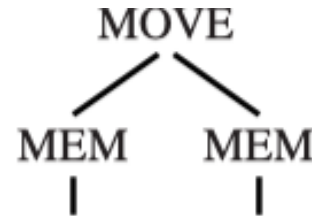
- **Largest tile:** the one with the most nodes.



one node



two nodes

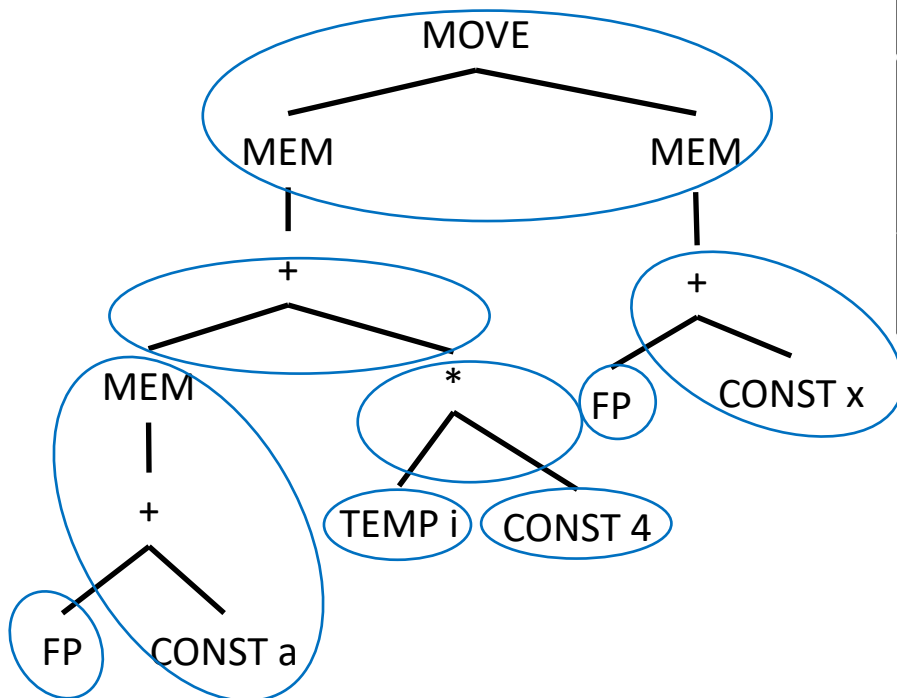


three nodes

- The instructions are generated in **reverse order**.
- If two tiles of equal size match at the root, then the choice between them is arbitrary.

Maximal Munch - Example

- Top-down strategy
- Cover the current node with the largest tile
- Repeat on subtrees
- **Example:** $a[i] := x$



Name	Effect	Trees
—	r_i	TEMP
ADD	$r_i \leftarrow r_j + r_k$	$\begin{array}{c} + \\ \swarrow \quad \searrow \end{array}$
MUL	$r_i \leftarrow r_j \times r_k$	$\begin{array}{c} * \\ \swarrow \quad \searrow \end{array}$
SUB	$r_i \leftarrow r_j - r_k$	$\begin{array}{c} - \\ \swarrow \quad \searrow \end{array}$
DIV	$r_i \leftarrow r_j / r_k$	$\begin{array}{c} / \\ \swarrow \quad \searrow \end{array}$
ADDI	$r_i \leftarrow r_j + c$	$\begin{array}{c} + \\ \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
SUBI	$r_i \leftarrow r_j - c$	$\begin{array}{c} - \\ \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
LOAD	$r_i \leftarrow M[r_j + c]$	$\begin{array}{c} \text{MEM} \quad \text{MEM} \quad \text{MEM} \quad \text{MEM} \\ \quad \quad \quad \\ + \quad + \quad \text{CONST} \quad \text{CONST} \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
STORE	$M[r_j + c] \leftarrow r_i$	$\begin{array}{c} \text{MOVE} \quad \text{MOVE} \quad \text{MOVE} \quad \text{MOVE} \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{MEM} \quad \text{MEM} \quad \text{MEM} \quad \text{MEM} \\ \quad \quad \quad \\ + \quad + \quad \text{CONST} \quad \text{CONST} \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{CONST} \quad \text{CONST} \end{array}$
MOVEM	$M[r_j] \leftarrow M[r_i]$	$\begin{array}{c} \text{MOVE} \\ \swarrow \quad \searrow \\ \text{MEM} \quad \text{MEM} \\ \quad \end{array}$

Maximal Munch - Implementation

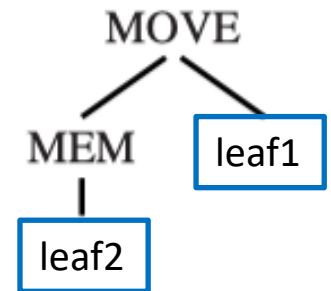
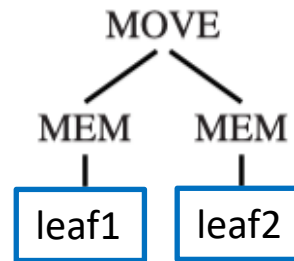
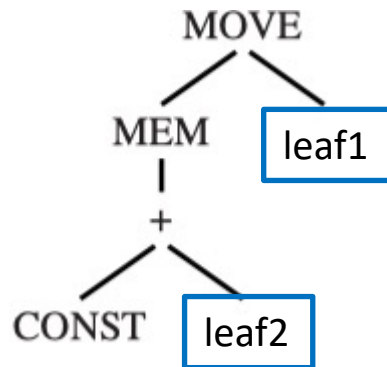
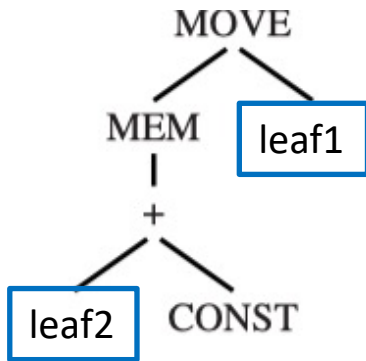
- Two recursive functions:
 - *munchStm* for statements
 - *munchExp* for expressions
- Each clause of *munchExp* will match one tile.
- The clauses are ordered in order of tile preference (largest tiles first)

Outline

- Algorithms for Instruction Selection
 - Maximal Munch
 - **Dynamic Programming**
 - Tree Grammar
- CISC Machines
- Instruction Selection for the Tiger Compiler

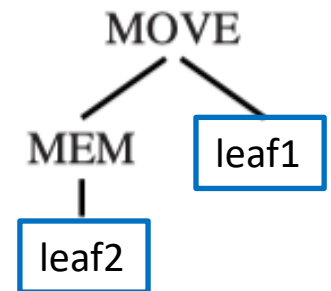
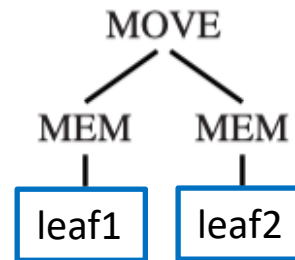
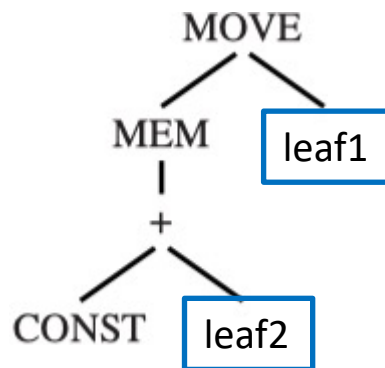
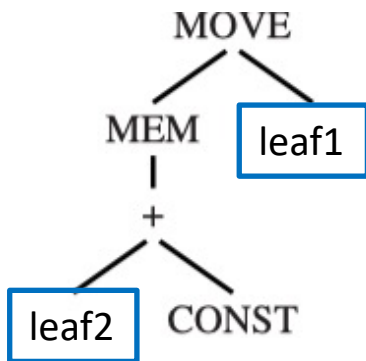
Find Optimum Tiling

- **Maximal munch**: always finds an **optimal** tiling, but not necessarily an optimum.
 - It works top-down
- How to find the optimum tiling?
- Brute force?



Dynamic Programming

- **Dynamic programming**: can find the **optimum** based on the optimum solution of each subproblem
 - It works bottom-up
 - Assign a **cost** to every node in the tree.
 - **The cost of node x** (denoted as $f(x)$) is the cost of the best tiling that covers the subtree rooted at x .
 - $f(x) = \min_{\forall \text{ tile } t \text{ covering } x} (c_t + \sum_{\forall \text{ leaf } i \text{ of } t} f(i))$
 - c_t refers to the cost of tile t .



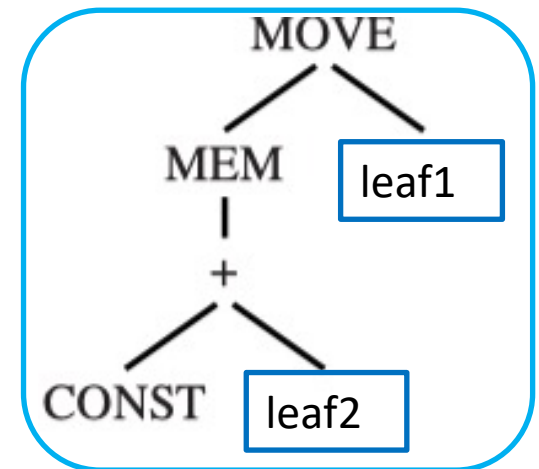
Dynamic Programming - Details

Given the IR Tree with root node n

- First, the costs of all the children (and grandchildren, etc.) of node n are found recursively.
- Then, each **tree pattern** (tile kind) is matched against node n
- Each tile has zero or more **leaves**, where subtrees can be attached.
- For each tile t of cost c_t that matches at node n , the cost of matching tile t is: ($f(i)$ has already been computed)

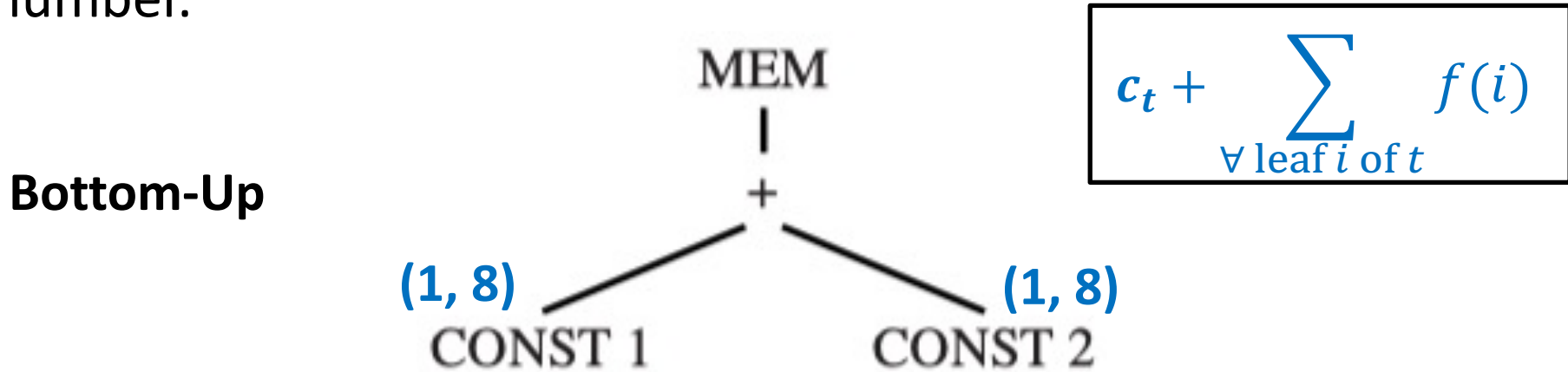
$$c_t + \sum_{\forall \text{ leaf } i \text{ of } t} f(i)$$

- The tree pattern leading to minimum cost is chosen



Dynamic Programming - Example

(a, b): a is the minimum cost, b is the corresponding pattern number.



Consider **CONST** node:

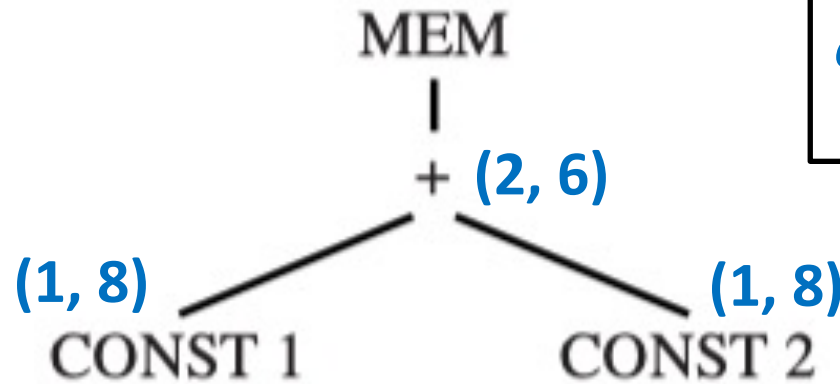
- The only tile matching CONST is the ADDI instruction with cost 1.
- There is no leaves in pattern (8).

Pattern (Tile)	Cost	Leaves Cost	Total
(8) CONST	1	0	1

Dynamic Programming - Example

(a, b): a is the minimum cost, b is the corresponding pattern number.

Bottom-Up



$$c_t + \sum_{\text{all leaves } i \text{ of } t} c_i$$

Consider + node:

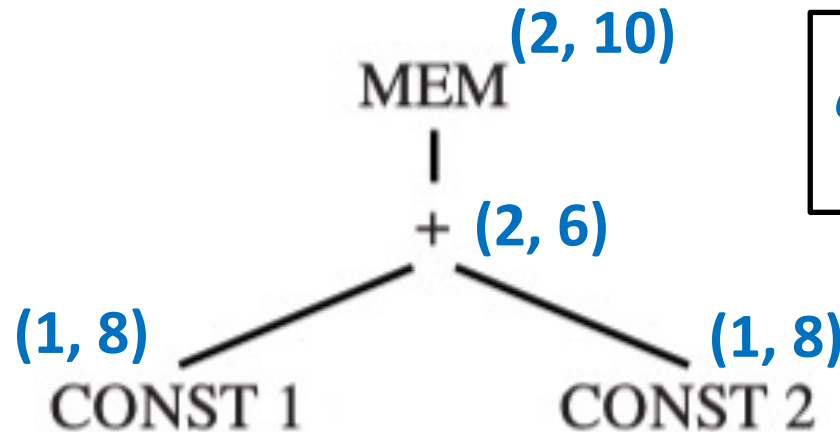
- Several tree patterns match the “+” node.

Pattern (Tile)	Cost	Leaves Cost	Total
(2) +(e1, e2)	1	2	3
(6) +(CONST, e1)	1	1	2
(7) +(e1, CONST)	1	1	2

Dynamic Programming - Example

(a, b): a is the minimum cost, b is the corresponding pattern number.

Bottom-Up



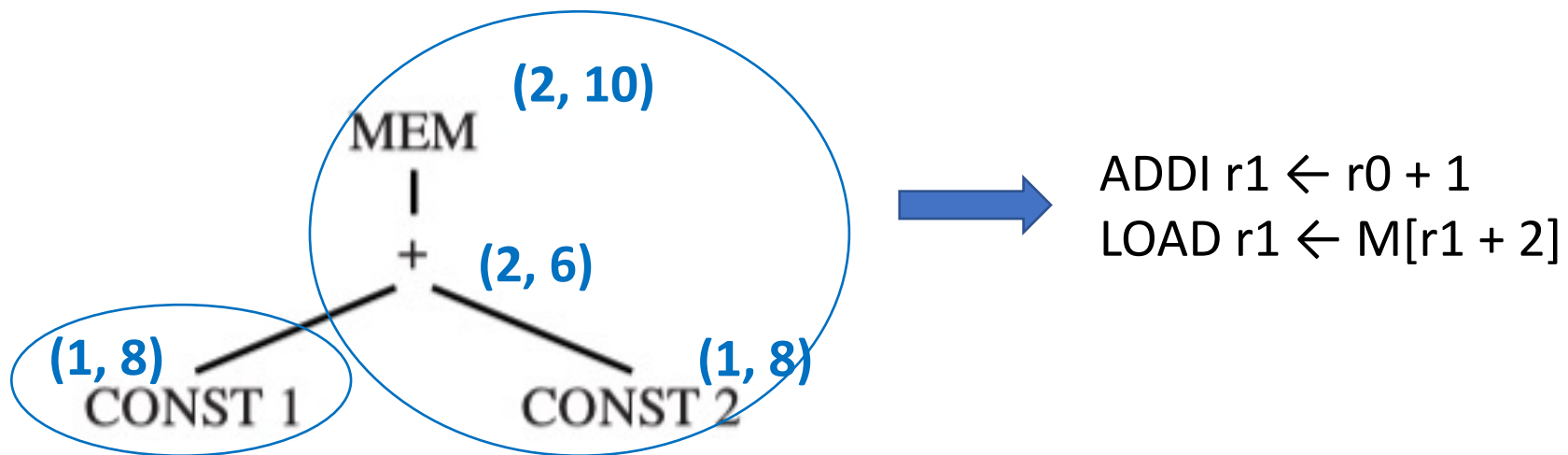
$$c_t + \sum_{\text{all leaves } i \text{ of } t} c_i$$

Consider MEM node:

Pattern (Tile)	Cost	Leaves Cost	Total
(13) MEM(e1)	1	2	3
(10) MEM(+ (e1, CONST))	1	1	2
(11) MEM(+ (CONST, e1))	1	1	2

Dynamic Programming – Instruction Emission

- Once the cost of the root node (and thus the entire tree) is found, the **instruction emission** phase begins. The algorithm is as follows:
- **Emission(node n):**
 - For each **leaf li** of the tile selected at node **n**, perform **Emission(li)**.
 - Emit the instruction matched at node **n**.



Fast Matching

Maximal munch and the dynamic-programming algorithm examine all the tiles that match a node.

- A tile matches if each non-leaf node of the tile is labeled with the same operator (MEM, CONST, etc.) as the corresponding node of the tree.
- To match a tile at node **n** of the tree, the label at **n** can be used in a case statement:

```
match(n) {  
  switch (label(n)) {  
    case MEM: ...  
    case BINOP: ...  
    case CONST: ...  
  }
```

- Goal: each node in an IR tree will not be looked twice

Efficiency of Tiling Algorithms

How expensive are maximal munch and dynamic programming?

- Suppose:
 - K : the average matching tile contains K non-leaf (labeled) nodes.
 - N : the number of nodes in the input tree
 - K' : the largest number of nodes that ever need to be examined to see which tiles match at a given subtree.
 - T' : the number of different patterns (tiles) that match at each tree node, on average
- Maximal munch: proportional to $(K' + T') N / K$
- Dynamic programming: proportional to $(K' + T') N$
- K , K' and T' are constant, the running time of all of these algorithms are linear.

Outline

- Algorithms for Instruction Selection
 - Maximal Munch
 - Dynamic Programming
 - **Tree Grammar (了解)**
- CISC Machines
- Instruction Selection for The Tiger Compiler

Tree Grammars

- For machines with **complex instruction sets** and **several classes of registers** and **addressing modes**:
 - More tree patterns and more complicated tree patterns.
 - Manually implementing the tiling algorithm becomes tedious and error-prone
- **Idea: instruction selector generators**
 - Define tree patterns in a separate specification
 - Use a generic tree pattern matching algorithm to compute tiling
- **Solution:**
 - Use a **tree grammar** (a special context-free grammar) to describe tree patterns
 - Reduce instruction selection to a parsing problem
 - Use a generalization of the dynamic programming algorithm for the parsing

Name	Effect	Trees
—	r_i	TEMP
ADD	$d_i \leftarrow d_j + d_k$	$ \begin{array}{c} d + \\ \swarrow \quad \searrow \\ d \quad \quad d \\ \swarrow \quad \searrow \\ d * \\ \swarrow \quad \searrow \\ d \quad \quad d \end{array} $
MUL	$d_i \leftarrow d_j \times d_k$	$ \begin{array}{c} d * \\ \swarrow \quad \searrow \\ d \quad \quad d \end{array} $
SUB	$d_i \leftarrow d_j - d_k$	$ \begin{array}{c} d - \\ \swarrow \quad \searrow \\ d \quad \quad d \end{array} $
DIV	$d_i \leftarrow d_j / d_k$	$ \begin{array}{c} d / \\ \swarrow \quad \searrow \\ d \quad \quad d \end{array} $
ADDI	$d_i \leftarrow d_j + c$	$ \begin{array}{c} d + \\ \swarrow \quad \searrow \\ d \quad \text{CONST} \end{array} \quad \begin{array}{c} d + \\ \swarrow \quad \searrow \\ \text{CONST} \quad d \end{array} \quad \begin{array}{c} d \text{ CONST} \end{array} $
SUBI	$d_i \leftarrow d_j - c$	$ \begin{array}{c} d - \\ \swarrow \quad \searrow \\ d \quad \text{CONST} \end{array} $
MOVEA	$d_j \leftarrow a_i$	d a
MOVED	$a_j \leftarrow d_i$	a d
LOAD	$d_i \leftarrow M[a_j + c]$	$ \begin{array}{c} d \text{ MEM} \\ \\ + \\ \swarrow \quad \searrow \\ a \quad \text{CONST} \end{array} \quad \begin{array}{c} d \text{ MEM} \\ \\ + \\ \swarrow \quad \searrow \\ \text{CONST} \quad a \end{array} \quad \begin{array}{c} d \text{ MEM} \\ \\ \text{CONST} \end{array} \quad \begin{array}{c} d \text{ MEM} \\ \\ a \end{array} $
STORE	$M[a_j + c] \leftarrow d_i$	$ \begin{array}{c} \text{MOVE} \\ \swarrow \quad \searrow \\ \text{MEM} \quad d \\ \quad \quad \\ + \quad \quad + \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ a \quad \text{CONST} \quad \text{CONST} \quad a \end{array} \quad \begin{array}{c} \text{MOVE} \\ \swarrow \quad \searrow \\ \text{MEM} \quad d \\ \quad \quad \\ + \quad \quad + \\ \swarrow \quad \searrow \quad \swarrow \quad \searrow \\ \text{CONST} \quad a \quad \text{CONST} \quad a \end{array} \quad \begin{array}{c} \text{MOVE} \\ \swarrow \quad \searrow \\ \text{MEM} \quad d \\ \quad \quad \\ \text{CONST} \quad a \end{array} \quad \begin{array}{c} \text{MOVE} \\ \swarrow \quad \searrow \\ \text{MEM} \quad d \\ \quad \quad \\ \text{CONST} \quad a \end{array} $
MOVEM	$M[a_j] \leftarrow M[a_i]$	$ \begin{array}{c} \text{MOVE} \\ \swarrow \quad \searrow \\ \text{MEM} \quad \text{MEM} \\ \quad \quad \\ a \quad \quad a \end{array} $

The brain-damaged version of *Jouette*:

- **a** registers for addressing
- **d** registers for “data”

The root and leaves of each tile must be marked with **a** or **d**.

Tree Grammars

- The corresponding tree grammar will have nonterminals:
 - s : for statements
 - a : for expressions calculated into an a register
 - d : for expressions calculated into a d register
- The grammar rules for the **LOAD**, **MOVEA** and **MOVED** instructions might look like this:

$d \rightarrow MEM(+ (a, CONST))$

$d \rightarrow MEM(+ (CONST, a))$

$d \rightarrow MEM(CONST)$

$d \rightarrow MEM(a)$

$d \rightarrow a$

$a \rightarrow d$

MOVEA	$d_j \leftarrow a_i$	d a
MOVED	$a_j \leftarrow d_i$	a d

LOAD	$d_i \leftarrow M[a_j + c]$	d MEM	d MEM	d MEM	d MEM
		+	+	CONST	a
		/ \	/ \		
		a CONST	CONST a		

Tree Grammars

- Such a grammar is highly ambiguous: there are many different instruction sequences implementing the same expression.
- The parsing techniques described in Chapter 3 are not very useful.
- However, a generalization of the dynamic-programming algorithm works quite well:
 - The dynamic programming algorithm must keep track, for each node, of the min-cost match as an *a* register and as a *d* register
 - The minimum-cost match at each node for *each nonterminal of the grammar* is computed.

Outline

- Algorithms for Instruction Selection
- **CISC Machines**
- Instruction Selection for the Tiger Compiler

RISC machine	CISC machine
32 registers	
only one class of integer/pointer register	
arithmetic operations only between registers	
“three-address” instructions of the form $r1 \leftarrow r2 \oplus r3$	
load and store instructions with only the $M[\text{reg}+\text{const}]$ addressing mode	
Every instruction exactly 32 bits long	
one result or effect per instruction	

RISC machine	CISC machine
32 registers	few registers (16, or 8, or 6)
only one class of integer/pointer register	registers divided into different classes, with some operations available only on certain registers;
arithmetic operations only between registers	arithmetic operations can access registers or memory through "addressing modes";
"three-address" instructions of the form $r1 \leftarrow r2 \oplus r3$	"two-address" instructions of the form $r1 \leftarrow r1 \oplus r2$;
load and store instructions with only the $M[\text{reg}+\text{const}]$ addressing mode	several different addressing modes;
Every instruction exactly 32 bits long	variable-length instructions, formed from variable-length opcode plus variable-length addressing modes;
one result or effect per instruction	instructions with side effects such as "autoincrement" addressing modes.

Problems and Solutions of CISC

- **Few registers:**
 - **Solution:** generate **TEMP** nodes freely, and assume that the register allocator will do a good job.
- **Classes of registers:**
 - Multiply on Pentium: left operand must be **eax**, the high-order bits of the result are put into **edx**
 - The high-order bits are useless to a Tiger program
 - **Solution:** move the operands and result explicitly.
 - Example: to implement **t1** $\leftarrow$ **t2** $\times$ **t3**

mov **eax**, **t2** **eax** $\leftarrow$ **t2**

mul **t3** **eax** $\leftarrow$ **eax** $\times$ **t3**; **edx** $\leftarrow$ garbage

mov **t1**, **eax** **t1** $\leftarrow$ **eax**

Problems and Solutions of CISC

- **Two-address instructions:**

- The destination register must be the same as the first source register
- **Solution:** Adding extra move instructions
- Example: to implement $t1 \leftarrow t2 + t3$

`mov t1,t2` $t1 \leftarrow t2$

`add t1, t3` $t1 \leftarrow t1 + t3$

- Then we hope that the register allocator will be able to allocate $t1$ and $t2$ to the same register, so that the `mov` instruction will be deleted.

Problems and Solutions of CISC

- **Arithmetic operations can address memory:**

- The instruction selection phase turns every **TEMP** node into a “register” reference. Many of these “registers” will actually turn out to be memory locations.
- **Solution:** Fetch all the operands into registers before operating and store them back to memory afterwards.
- Example: these two sequences compute the same thing:

`mov eax, [ebp - 8]`

`add eax, ecx`

`mov [ebp - 8], eax`

`add [ebp - 8], ecx`

- The sequence on the right is more concise, but the two sequences are equally fast.
- The sequence on the left has one significant disadvantage: It trashes the value in register **eax**

Problems and Solutions of CISC

- **Several addressing modes:**

- An addressing mode that accomplishes six things typically takes six steps to execute.
- two advantages:
 - They “trash” fewer registers
 - Shorter instruction encoding
- With some work, tree-matching instruction selection can be made to select CISC addressing mode, but programs can be just as fast using the simple RISC-like instructions

- **Variable-length instructions:**

- Not really a problem for the compiler;
- Once the instructions are selected, it is a trivial (though tedious) matter for the assembler to emit the encodings.

Problems and Solution of CISC

- **Instructions with side effects:**

- Problem: some machines have an “autoincrement” memory fetch instruction whose effect is:

$$r2 \leftarrow M[r1]; r1 \leftarrow r1 + 4$$

- Difficult to model using tree patterns, since it **produces two results**.
- There are three solutions:
 - Ignore the autoincrement instructions, do not use them, and hope they go away.
 - Try to match special idioms in an ad hoc way, within the context of a tree pattern-matching code generator.
 - Use a different instruction algorithm entirely, one based on DAG patterns instead of tree patterns.

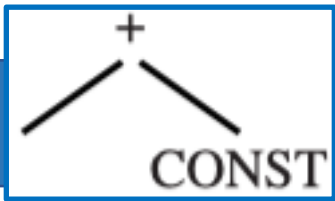
Algorithms for Instruction Selection

- Algorithms for optimal tilings are simpler than optimum tilings.
- **For CISC**, the difference between optimum and optimal tilings is noticeable.
 - Some instructions accomplish several operations each.
- **For RISC**, there is usually no difference at all between optimum and optimal tilings.
 - The tiles are small and of uniform cost.
- Thus, for RISC, the simpler tiling algorithms suffice.

Outline

- Algorithms for Instruction Selection
- CISC Machines
- **Instruction Selection for the Tiger Compiler**

Instruction Selection for Tiger



- In a tree tiled by instruction patterns, the root of each tile will correspond to some intermediate result held in a register.
- Which register to use?
- The task of **register allocation** is to assign register-numbers to each such node.
- Many aspects of register allocation are independent of the particular target-machine instruction set.
- Should come either before or after instruction selection
 - Before instruction selection: it is not even known which tree nodes will need registers to hold their results
 - cannot be very accurate.
 - We will do register allocation after instruction selection.

Abstract Assembly Language Instructions

```
/* assem.h*/
typedef struct {Temp_labelList labels;} *AS_targets;
AS_targets AS_Targets(Temp_labelList labels);

typedef struct {
    enum {I_OPER, I_LABEL, I_MOVE} kind;
    union {
        struct {string assem; Temp_tempList dst, src; AS_targets
jumps;} OPER;
        struct {string assem; Temp_label label;} LABEL;
        struct {string assem; Temp_tempList dst, src;} MOVE;
    } u;
} *AS_instr; /* assembly language instruction without register
assignments */

AS_instr AS_Oper(string a, Temp_tempList d, Temp_tempList s,
AS_targets j);
AS_instr AS_Label(string a, Temp_label label);
AS_instr AS_Move(string a, Temp_tempList d, Temp_tempList s);
void AS_print(FILE *out, AS_instr i, Temp_map m);
...
```

Abstract Assembly Language Instructions

```
typedef struct {Temp_labelList labels;} *AS_targets;
typedef struct {
    enum {I_OPER, I_LABEL, I_MOVE} kind;
    union {
        struct {string assem; Temp_tempList dst, src; AS_targets
jumps;} OPER;
        struct {string assem; Temp_label label;} LABEL;
        struct {string assem; Temp_tempList dst, src;} MOVE;
    } u;
} *AS_instr;
```

- An **OPER** holds:
 - **assem**: an assembly language instruction
 - **src**: a list of operand registers (may be empty)
 - **dst**: a list of result registers (may be empty)
 - **jumps**: a list of “target” labels to which they may jump
 - Operations that always fall through to the next instruction have jumps=NULL

Abstract Assembly Language Instructions

```
typedef struct {Temp_labelList labels;} *AS_targets;
typedef struct {
    enum {I_OPER, I_LABEL, I_MOVE} kind;
    union {
        struct {string assem; Temp_tempList dst, src; AS_targets
jumps;} OPER;
        struct {string assem; Temp_label label;} LABEL;
        struct {string assem; Temp_tempList dst, src;} MOVE;
    } u;
} *AS_instr;
```

- A LABEL is a point in a program to which jumps may go.
 - assem: how the label will look in the assembly language program
 - label: which label symbol was used

Abstract Assembly Language Instructions

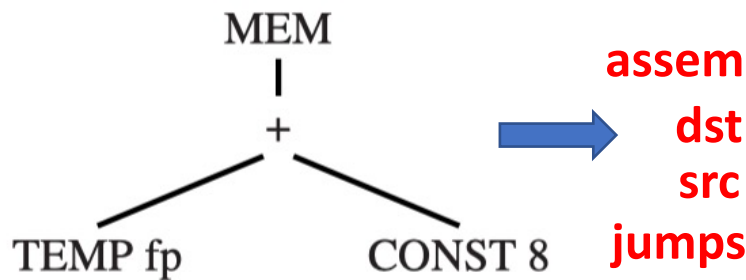
```
typedef struct {Temp_labelList labels;} *AS_targets;
typedef struct {
    enum {I_OPER, I_LABEL, I_MOVE} kind;
    union {
        struct {string assem; Temp_tempList dst, src; AS_targets
jumps;} OPER;
        struct {string assem; Temp_label label;} LABEL;
        struct {string assem; Temp_tempList dst, src;} MOVE;
    } u;
} *AS_instr;

void AS_print(FILE *out, AS_instr i, Temp_map m);
```

- A **MOVE** is like an **OPER**, but must perform only data transfer
- **AS_print(f, i, m)** formats an assembly instruction **i** as a string and prints it to the file **f**:
 - **m**: a *temp mapping* that tells the register assignment of every temp

Machine-Independence

- The `AS_instr` type is independent of the chosen target-machine assembly language.
- Use *Jouette* assembly language for illustration



```
AS_Oper(  
  "LOAD 'd0 <- M['s0+8]",  
  Temp_TempList(Temp_newtemp(), NULL),  
  Temp_TempList(T_Temp(F_FP()), NULL),  
  NULL)
```

Jouette assembly language

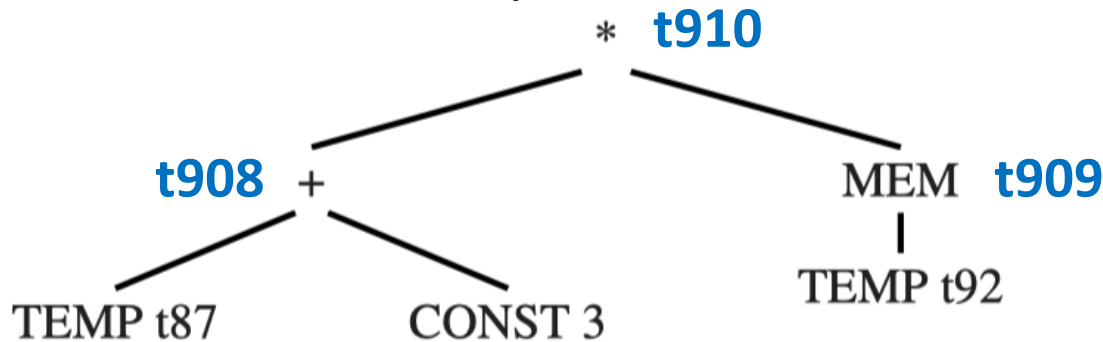
- The actual assembly language of *Jouette*, after register allocation, might be:

LOAD r1 <- M[r27+8]

- The `Assem` instruction does not know about register assignments. It refers to the first source register as ``s0` and the destination register as ``d0`

Machine-Independence

- Another example:



assem	dst	src
ADDI `d0 <- `s0 + 3	t908	t87
LOAD `d0 <- M[`s0 + 0]	t909	t92
MUL `d0 <- `s0 * `s1	t910	t908, t909

where t908, t909 and t910 are temporaries newly chosen by the instruction selector.

- After register allocation, the assembly language might look like:

```
ADDI    r1 <- r12+3
LOAD    r2 <- M[r13+0]
MUL     r1 <- r1 * r2
```

Two-Address Instructions

- Some machines have arithmetic instructions with two operands, where one of the operands is both a source and a target.
- For example, the instruction `add t1, t2`, which has the effect of `t1 <- t1 + t2`, can be described as

assem	dst	src
add `d0, `s1	t1	t1, t2

where 's0 is implicitly, but not explicitly, mentioned in the assem string.

Procedure Calls

- Procedure calls: `EXP(CALL(f, args))`, Function calls: `MOVE(TEMP t, CALL(f, args))`
- These trees can be matched by the tiles such as

```
case EXP(CALL(e, args)) {  
    Temp_temp r = munchExp(e);  
    Temp_tempList l = munchArgs(0, args);  
    emit(AS_Oper("CALL 's0\n", calldefs, L(r, l), NULL));}
```

- `munchArgs` generates code to move all the arguments to their correct positions (in registers or in memory).
- A `CALL` is expected to “trash” certain registers (caller-save registers, return-address register, return-value register). This list of calldefs should be listed as “destinations” of the CALL.
- In general, any instruction that has the side effect of writing to another register requires this treatment.

If There's No Frame Pointer

Use frame pointer: at each procedure call,

- the stack pointer reg is copied to the frame pointer reg
- the stack pointer is incremented by the size of the new frame

Virtual frame pointer

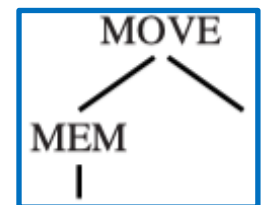
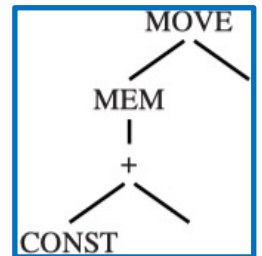
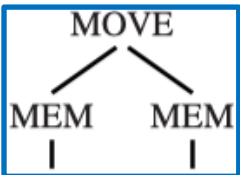
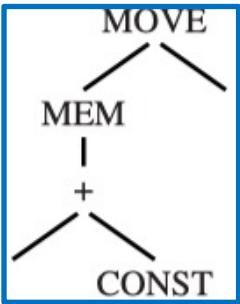
- Saves time (no copy instruction) and space (one more register usable for other purposes)
- The `codegen` function must replace any reference to $FP + k$ with $SP + k + fs$, where fs is the frame size.

Homework

- **9.1**

Maximal Munch - Implementation

```
static void munchStm(T_stm s) {
    switch(s->kind) {
        case T_MOVE: {
            T_exp dst = s->u.MOVE.dst, src = s->u.MOVE.src;
            if (dst->kind==T_MEM) {
                if (dst->u.MEM->kind==T_BINOP && dst->u.MEM->u.BINOP.op==T_plus
                    && dst->u.MEM->u.BINOP.right->kind==T_CONST){
                    T_exp e1 = dst->u.MEM->u.BINOP.left, e2=src;
                    /* MOVE(MEM(BINOP(PLUS, e1, CONST(i))), e2) */
                    munchExp(e1); munchExp(e2); emit("STORE");
                } else if (dst->u.MEM->kind==T_BINOP
                    && dst->u.MEM->u.BINOP.op==T_plus
                    && dst->u.MEM->u.BINOP.left->kind==T_CONST) {
                    T_exp e1 = dst->u.MEM->u.BINOP.right, e2=src;
                    /* MOVE(MEM(BINOP(PLUS, CONST(i), e1)), e2) */
                    munchExp(e1); munchExp(e2); emit("STORE");
                } else if (src->kind==T_MEM) {
                    T_exp e1 = dst->u.MEM, e2=src->u.MEM;
                    /* MOVE(MEM(e1), MEM(e2)) */
                    munchExp(e1); munchExp(e2); emit("MOVEM");
                } else {
                    T_exp e1 = dst->u.MEM, e2=src;
                    /* MOVE(MEM(e1), e2) */
                    munchExp(e1); munchExp(e2); emit("STORE");
                }
            }
        }
    }
}
```



...

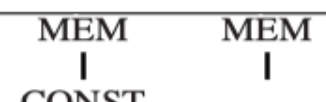
Maximal Munch - Implementation

```
static void munchStm(T_stm s) {
    switch(s->kind) {
        case T_MOVE: {
            T_exp dst = s->u.MOVE.dst, src = s->u.MOVE.src;
            if (dst->kind==T_MEM) {
                ...
            } else if (dst->kind==T_TEMP) {
                T_exp e2=src;
                /* MOVE(TEMP i , e2) */
                munchExp(e2); emit("ADD");
            }
            else assert(0); /* destination of MOVE must be MEM or TEMP */
            break;
        }
        case T_JUMP: ...
        case T_CUMP: ...
        case T_NAME: ...
        ...
    }
}
```

Maximal Munch - Implementation

```
static void munchStm(T_stm s)    MunchExp
MEM(BINOP(PLUS, e1, CONST(i))) ⇒ munchExp(e1); emit("LOAD");
MEM(BINOP(PLUS, CONST(i), e1)) ⇒ munchExp(e1); emit("LOAD");
MEM(CONST(i)) ⇒ emit("LOAD");
MEM(e1) ⇒ munchExp(e1); emit("LOAD");
BINOP(PLUS, e1, CONST(i)) ⇒ munchExp(e1); emit("ADDI");
BINOP(PLUS, CONST(i), e1) ⇒ munchExp(e1); emit("ADDI");
CONST(i) ⇒ munchExp(e1); emit("ADDI");
BINOP(PLUS, e1, CONST(i)) ⇒ munchExp(e1); emit("ADD");
TEMP(t) ⇒ {}
```

PROGRAM 9.4. Summary of the C function `munchStm`.

ADDI	$r_i \leftarrow r_j + c$		CONST
LOAD	$r_i \leftarrow M[r_j + c]$		

Abstract Assembly Language Instructions

```
/* temp.h */  
...  
typedef struct Temp_map_ *Temp_map;  
Temp_map Temp_empty(void); /* create a new, empty map */  
Temp_map Temp_layerMap(Temp_map over, Temp_map under);  
void Temp_enter(Temp_map m, Temp_temp t, string s);  
string Temp_look(Temp_map m, Temp_temp t);  
  
Temp_map Temp_name(void);
```

- A **Temp_map** is just a table whose keys are **Temp_temps** and whose bindings are **strings**.
- One mapping can be layered over another.
 - if $\sigma_3 = \text{layerMap}(\sigma_1, \sigma_2)$, this means that $\text{look}(\sigma_3, t)$ will first try $\text{look}(\sigma_1, t)$, and if that fails it will try $\text{look}(\sigma_2, t)$

Producing Assembly Instructions

- `emit("STORE");`
- Now, how to “munch” `Tree` expressions into `Assem` instructions?
- The function `munchStm` and `munchExp` will produce `Assem` instructions, bottom-up, as side effects. `munchExp` returns the temporary in which the result is held.

```
static Temp_temp munchExp(T_exp e);  
static void munchStm(T_stm s);
```

Producing Assembly Instructions

```
static Temp_temp munchExp(T_exp e) {
switch( e )
case MEM(BINOP(PLUS,e1,CONST (i))): {
    Temp_temp r = Temp_newtemp();
    emit(AS_Oper(
        "LOAD 'd0 <- M['s0+" + i + "']\n",
        L(r,NULL),
        L(munchExp(e1),NULL),
        NULL)
    );
    return r;
}
case MEM(BINOP(PLUS,CONST (i),e1)): {
    Temp_temp r = Temp_newtemp();
    emit(AS_Oper(
        "LOAD 'd0 <- M['s0+" + i + "']\n",
        L(r,NULL),
        L(munchExp(e1),NULL),
        NULL)
    );
    return r;
}
...
}
```

Producing Assembly Instructions

```
static void munchStm(T_stm s) {
    switch ( s )
    case MOVE(MEM(BINOP(PLUS,e1,CONST(i))),e2):
        emit(AS_Oper(
            "STORE M['s0+' + i + '] <- 's1\n",
            NULL,
            L(munchExp(e1), L(munchExp(e2), NULL)),
            NULL)
        );
    case MOVE(MEM(BINOP(PLUS,CONST (i),e1)),e2):
        emit(AS_Oper(
            "STORE M['s0+' + i + '] <- 's1\n",
            NULL,
            L(munchExp(e1), L(munchExp(e2), NULL)),
            NULL)
        );
    ...
    case LABEL(lab):
        emit(AS_Label(Temp_labelstring(lab) + ":\n", lab));
    ...
}
```

Producing Assembly Instructions

```
/* codegen.c */  
...  
static AS_instrList iList=NULL, last=NULL;  
static void emit(AS_instr inst) {  
    if (last!=NULL)  
        last = last->tail = AS_InstrList(inst,NULL);  
    else  
        last = iList = AS_InstrList(inst,NULL);  
}
```

- The `emit` function just accumulates a list of instructions to be returned later.